

Adaptive Shot Management for Quantum Circuit-Cutting Services

Alessandro Bocci^{1*}, Giuseppe Bisicchia¹, Ernesto Pimentel², and Antonio Brogi¹

¹ University of Pisa, Pisa, Italy

`alessandro.bocci@unipi.it`, `giuseppe.bisicchia@phd.unipi.it`,
`antonio.brogi@unipi.it`

² University of Malaga, Malaga, Spain
`epimentel@uma.es`

Abstract. Quantum computing is increasingly accessed through remote execution services, where users submit circuits to cloud-based quantum backends and receive classical results from repeated measurements. Since each quantum measurement returns a probabilistic outcome, a circuit must usually be executed many times; each execution is called a *shot*. Shots are therefore a key service-level resource: they affect statistical quality, backend usage, queueing, latency, and cost.

This paper addresses shot management in quantum circuit cutting, a technique that evaluates circuits larger than the available hardware by decomposing them into smaller subcircuits, executing them separately, and classically recombining their results. From a service-oriented quantum perspective, circuit cutting can be viewed as a structured quantum service involving cut selection, subcircuit execution, and reconstruction. Existing work mainly focuses on cutting and reconstruction, while the execution-level problem of controlling how many shots to spend on each generated subcircuit remains less explored.

We present an adaptive execution-control layer for quantum circuit-cutting services. Given a cut circuit and a maximum shot budget, the layer executes each subcircuit in small batches, monitors the evolution of their empirical output distribution, and stops execution once a configurable stabilization criterion is satisfied. It is designed as a modular component placed between cutting and sewing, so that adaptive shot control can be added without changing the decomposition or reconstruction procedures.

We evaluate the resulting workflow on 70 quantum circuits, four Qiskit fake backends, and three budget tiers, comparing fixed-budget circuit cutting with adaptive execution. The results show that the adaptive workflow achieves error levels close to the corresponding full-budget cutting baselines in the evaluated configurations, while reducing shot consumption by more than 25% on average and reaching peak savings of about 67%.

* Corresponding author.

Keywords: Service-Oriented Quantum · Quantum Computing · Quantum Software Engineering · Shot optimization

1 Introduction

Quantum computing is moving from isolated laboratory systems to remotely accessible execution platforms. In this model, users submit quantum circuits to remote quantum processors or simulators and receive classical data collected from measured executions. This deployment model places quantum computing within the scope of service-oriented computing, but recent work argues for an even stronger view. The Service-Oriented Quantum (SOQ) paradigm proposes treating quantum capabilities as first-class services: autonomous, composable, interoperable, and governed by service-level concerns rather than hidden as low-level accelerators inside classical applications [10]. From this perspective, a quantum service should not only expose computational functionality. It should also manage constraints such as backend heterogeneity, qubit availability, queueing time, execution cost, and result fidelity.

This service-oriented view is especially important in the current noisy intermediate scale quantum (NISQ) era [19]. Available devices remain constrained by limited qubit counts, device noise, calibration drift, and finite execution budgets. As a consequence, the design of quantum services cannot stop at circuit submission and result retrieval. It also requires execution-control mechanisms able to decide how scarce quantum resources are consumed during a service invocation. In SOQ terms, this is part of making quantum services manageable and composable: a service should be able to expose and control the trade-off between resource usage and output quality.

A key resource in quantum execution is the number of *shots*. A quantum circuit is built from quantum bits, or *qubits*, transformed by quantum gates, and measured at the end of the execution. Unlike a deterministic classical program, one execution of a quantum circuit returns a sample from an underlying probability distribution. Therefore, the same circuit is executed repeatedly; each complete execution and measurement is called a *shot*. The observed frequencies over many shots form an empirical estimate of the circuit output distribution or of a scalar quantity derived from it, such as an observable expectation value. More shots usually improve statistical confidence, but they also consume backend time, increase queue usage, and may contribute directly to monetary cost or service quotas. Shot control is therefore not only a quantum-statistical issue, but also a service-management issue [1,2]. In this paper, we focus on shot consumption as the primary resource metric because it is the execution-level resource directly controlled by the proposed layer. We do not assume that shot savings always translate proportionally into end-to-end service savings: on real cloud platforms, batching may introduce additional submissions, network overheads, or queue interactions. Our evaluation therefore isolates the question of whether a circuit-cutting workflow can consume fewer quantum shots while preserving the

final observable estimate; provider-specific latency, scheduling, and cost effects are left for future work.

Circuit cutting addresses another fundamental limitation of near-term devices: some circuits require more qubits than a target backend provides. The idea is to split a large circuit into smaller subcircuits, execute those subcircuits separately, and then classically *sew* their results to estimate the output of the original circuit [16,21,17]. In a service-oriented architecture, circuit cutting can be seen as a composite quantum service: a cut-selection component decomposes the input circuit, execution components run the generated subcircuits on quantum backends or simulators, and a reconstruction component combines the partial results. This decomposition improves feasibility, but it also turns one quantum job into a workflow of many smaller service invocations. The question then becomes how to govern the execution of these invocations under a limited shot budget.

Most circuit-cutting research focuses on cut placement, overhead reduction, and reconstruction quality. However, once a circuit has been cut, the workflow must also decide how to distribute the available shot budget among the generated subcircuits or execution variants. We refer to this decision as a shot-allocation policy. Existing policies [4] typically make this decision before execution, using static properties of the cut workload, such as the number of subcircuits, the number of qubits, or gate-level structure. These policies are simple and predictable, but they treat the assigned budget as a fixed execution plan, without observing whether some subcircuits may require fewer shots than initially allocated. This motivates an adaptive execution-control layer that operates after the initial allocation and during subcircuit execution.

We address this problem by exploiting STABLESHOTS [7] within circuit-cutting workflows. STABLESHOTS executes a circuit in small shot batches and stops once the observed empirical distribution satisfies a configurable stabilization criterion. In our setting, STABLESHOTS is used as a modular execution-control layer for a circuit-cutting service: the circuit-cutting and sewing procedures remain unchanged, while each subcircuit is executed adaptively within an assigned maximum budget. We do not aim to define a complete SOQ platform; rather, we study a concrete adaptive-management mechanism that such a platform could use to reduce resource consumption while preserving result quality.

The paper makes three contributions. First, it treats subcircuit shot allocation as an execution-management problem within a service-oriented circuit-cutting workflow. Second, it places STABLESHOTS between cutting and sewing as an online controller rather than introducing a new stopping rule. Each execution unit keeps its own stopping state and maximum budget, while all local estimates contribute to the same sewn result. Local subcircuit stability therefore does not by itself imply stability after sewing, since reconstruction can amplify subcircuit-level errors. Neither cutting nor sewing is changed. Third, we compare the controller with fixed-budget cut execution policies inspired by [4]. For the evaluated observable, online stopping achieves error levels close to the corre-

sponding fixed-budget execution while reducing shot consumption by more than 25% on average, with peak savings of about 67%.

2 Background and Related Work

2.1 Quantum Computing

A quantum circuit is an ordered composition of operations, called gates, applied to qubits. Qubits are the quantum analogue of classical bits, but before measurement their state is generally represented by amplitudes over many possible classical outcomes. The final measurement converts the quantum state into a classical result. Because quantum measurement is probabilistic, repeated measurements of the same circuit can produce different bit strings.

A *shot* is one complete execution and measurement of a circuit. Running a circuit for n shots produces n samples from its output distribution. From these samples, one can estimate either the full probability distribution over bit strings or an *observable*, i.e., a numerical property computed from that distribution. In this paper, the methodology is formulated at the level of probability distributions, while the experiments report the error of a single observable expectation value because of the cutting backend used³.

This sampling view is important for service-oriented execution. In a quantum service, shots are the basic unit that consumes backend time and contributes to queue occupancy, cost, and statistical quality. A service that always consumes a fixed shot budget may be robust but inefficient; a service that stops too early may save resources but degrade result quality. The central question is therefore how to decide, during execution, whether additional shots are still useful.

2.2 Circuit Cutting

Circuit cutting replaces selected connections in a quantum circuit with local operations and classical post-processing, enabling large circuits to be evaluated on smaller devices [16]. The resulting workflow has three stages. First, a cutting procedure decomposes the original circuit into subcircuits that fit the target backend. Second, the subcircuits are executed, typically many times, to estimate their output behaviour. Third, a sewing procedure recombines the subcircuit estimates to approximate the result of the original circuit.

In this context, CUTQC [21] casts this process as a system-level workflow combining cut selection, subcircuit execution, and reconstruction of the original output distribution. Maximum-likelihood subcircuit tomography [17] improves reconstruction from finite-shot subcircuit data by projecting noisy estimates onto valid distributions. Other work studies randomized-measurement cutting [15], circuit knitting with classical communication [18], and optimal sampling overheads for wire and multi-gate cutting [8,20]. These approaches differ in the type

³ Moving from an empirical distribution to an expectation value simply requires computing a linear combination of the components of the probability distribution.

of cut, the reconstruction rule, and the overhead analysis, but they share a practical consequence: the runtime must orchestrate the execution of many subcircuits before sewing can be performed, and none of those approaches tackle the problem of determining the number of shots to assign to each subcircuit.

From a quantum-software-engineering perspective, circuit cutting shifts part of the challenge from circuit feasibility to execution orchestration. CUT&SHOOT [4] is the closest circuit-cutting workflow to our setting because it explicitly considers how shots are assigned after cutting. It studies fixed policies that distribute shots according to static workload properties, such as the number of generated subcircuits, the number of qubits, or the number of two-qubit gates. In our evaluation, we use two of these policies as fixed-budget baselines. SHOTQC [9] addresses the same sampling cost from another point in the workflow. It optimizes shot distribution and cut parameterization before execution. Our controller starts after the cut, reconstruction procedure, and maximum subcircuit budgets have been chosen, and decides how much of each budget is consumed at runtime. CUT&SHOOT therefore provides pre-execution allocation policies, SHOTQC optimizes the sampling plan and cutting formulation, and our approach controls execution online. The three mechanisms can be combined.

2.3 Shot Optimization and Adaptive Execution Control

Shot optimization asks how many repeated executions are needed to estimate a quantum circuit output with sufficient quality [5,6]. Many techniques target variational or adaptive algorithms, where shot counts are allocated using gradients, Hamiltonian structure, variance estimates, entropy, or learned policies [13,11,12,14]. These methods exploit algorithm-specific information and non-stationary optimization loops.

STABLESHOTS [7] instead targets static, non-variational circuits in a black-box setting. It executes a fixed circuit in batches of b shots and, after each batch, monitors how the accumulated empirical output distribution changes. Stability is assessed by comparing the current distribution with previous accumulated estimates over a lookback window of ℓ batches; if the total variation distance remains below a threshold ϵ for k consecutive checks, execution stops. The parameters therefore control the granularity of execution (b), the temporal window used to detect stabilization (ℓ), the tolerance for distributional change (ϵ), and the required persistence of stability (k). This black-box and static-circuit assumption makes STABLESHOTS suitable for cut subcircuits: once a cut is fixed, each subcircuit execution is itself a static sampling task. The main caveat is that local subcircuit stability need not imply global stability after sewing, since reconstruction can amplify subcircuit-level errors. For this reason, our evaluation considers the final sewn result rather than only local subcircuit convergence. We do not introduce a new stopping criterion. The contribution is to place an existing black-box rule inside a multi-subcircuit workflow, where each execution unit can stop independently but all local estimates feed the same sewn result. This separates runtime shot control from cut selection and reconstruction and makes shot consumption an explicit property of the composed service.

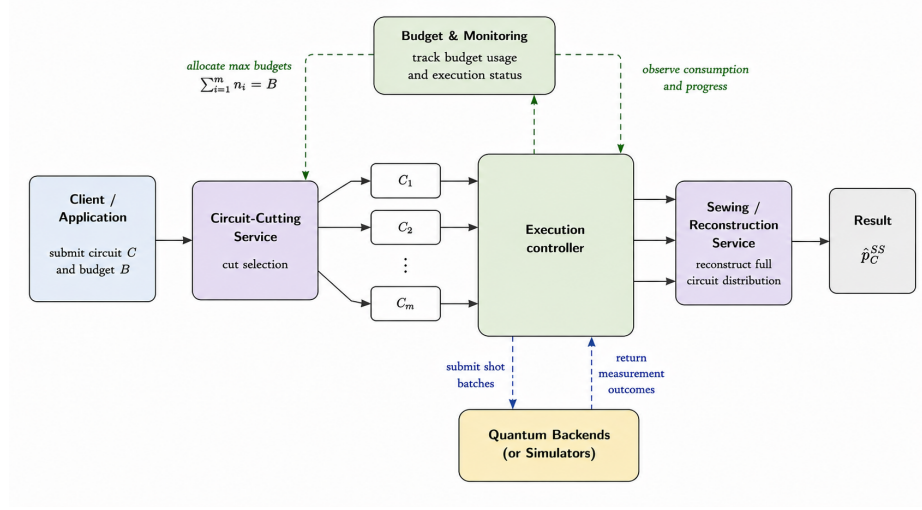


Fig. 1. Fixed-budget circuit-cutting service workflow and shot-allocation problem. A circuit is decomposed into m execution units, which collectively consume the complete allocated budget B , such that $\sum_{i=1}^m n_i = B$. Their estimates are subsequently reconstructed to produce the final result.

3 Adaptive Shot Control for Circuit-Cutting Services

The proposed approach is a lightweight execution-control layer for cut quantum circuits. It assumes that a circuit-cutting procedure and a sewing procedure are already available, and it does not modify either of them. Its role is to decide, during execution, whether each cut subcircuit still needs additional shots or whether its empirical estimate has stabilized enough to stop.

In a service-oriented implementation, the layer can be placed between a circuit-cutting component and a backend execution component. It receives subcircuits and maximum budgets from the cutting workflow, submits execution batches to the selected quantum backend or simulator, observes the accumulated outcomes, applies a stabilization test, and returns the subcircuit estimates to the sewing component. Figure 1 summarizes this modular placement. The key design choice is that execution control is separated from both cut selection and reconstruction. This separation makes the layer reusable with different circuit-cutting and sewing methods.

These design choices also map directly onto SOQ principles. The execution controller can be inserted between cutting and sewing without changing either component, which supports *composability*. Because it consumes streamed measurement outcomes and does not depend on a particular cutting algorithm or backend internals, the same interface supports *interoperability*. Maximum budgets, consumed-shot counts, and stopping conditions provide explicit controls for service *governance*, while the execution stage can be configured independently

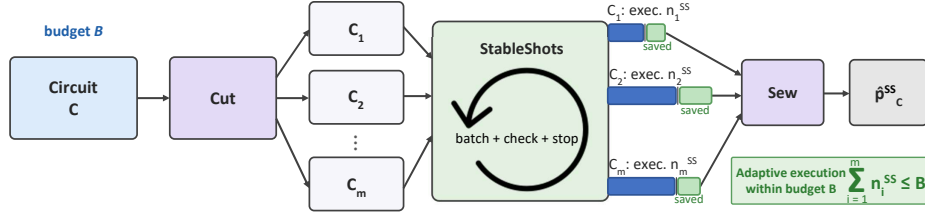


Fig. 2. Adaptive execution of the circuit-cutting workflow using STABLESHOTS. Each execution unit has a maximum allocation n_i , but may terminate after $n_i^{SS} \leq n_i$ shots when stabilization is detected. Consequently, the total consumed budget satisfies $\sum_{i=1}^m n_i^{SS} \leq B$, and the unused portion is saved.

within the budget set by the surrounding workflow, providing a degree of *autonomy*. Our evaluation measures shot consumption directly; latency and monetary cost depend on the provider and are not measured here.

3.1 Problem Statement

Given an input circuit C , circuit cutting decomposes it into a set of m subcircuits⁴ $\mathcal{C} = \{C_1, \dots, C_m\}$. A global shot budget B is fixed before execution, and a shot vector $\mathbf{n} = (n_1, \dots, n_m)$ must be assigned to the subcircuits such that

$$\sum_{i=1}^m n_i = B.$$

Each subcircuit C_i is then executed using n_i shots, producing an empirical estimate $\hat{p}_i$. The full-circuit output estimate $\hat{p}_C$ is finally obtained by sewing together the subcircuit estimates.

The key problem is how to choose and consume the allocation vector $\mathbf{n}$. This is non-trivial because different subcircuits may require different numbers of shots to obtain sufficiently stable output estimates. A uniform or purely static allocation can over-allocate shots to subcircuits that converge quickly and under-allocate shots to subcircuits whose output distributions require more samples. Existing policies address this issue by assigning shots according to static properties of the cut circuit. In contrast, this work investigates whether execution can be guided online by the observed stabilization behavior of each subcircuit.

3.2 Methodology

Figure 2 shows the proposed methodology. The input circuit C is first decomposed into m subcircuits $\mathcal{C} = \{C_1, \dots, C_m\}$ using a standard circuit-cutting procedure. The global shot budget B is fixed before execution, as in fixed-budget

⁴ We use *subcircuits* to denote all execution variants generated by cutting, each treated as an independent execution unit.

approaches, and each subcircuit C_i is associated with a static execution upper bound n_i such that $\sum_{i=1}^m n_i = B$. However, rather than preassigning and fully consuming a static shot vector, we apply STABLESHOTS independently to each subcircuit, allowing execution to terminate before reaching n_i when stability is detected.

For each subcircuit C_i , execution is performed in batches of b shots. After each batch, the accumulated empirical output distribution is updated and compared with previous estimates using a stabilization criterion. If the distribution is considered stable, the execution of C_i stops; otherwise, another batch is executed until either stabilization is reached or the maximum available budget is consumed. STABLESHOTS declares subcircuit C_i stable when, for k consecutive checks, the total variation distance between the current accumulated empirical distribution and the accumulated distributions observed over the last ℓ batches remains below a threshold ϵ . We denote by n_i^{SS} the number of shots actually used by STABLESHOTS for subcircuit C_i .

The reconstructed output distribution is then obtained by applying the usual sewing procedure to the subcircuit estimates:

$$\hat{p}_C^{SS} = \text{Sew} \left(\hat{p}_1^{(n_1^{SS})}, \dots, \hat{p}_m^{(n_m^{SS})} \right).$$

Since each subcircuit may stop before consuming its maximum allocation, the total number of executed shots satisfies

$$\sum_{i=1}^m n_i^{SS} \leq B, \quad \text{with } n_i^{SS} \leq n_i \ \forall i \in \{1, \dots, m\}.$$

The controller acts on empirical subcircuit distributions and can in principle feed sewing procedures that reconstruct distributions or observables. In our implementation, however, PennyLane’s `cut_circuit` transform returns the selected observable expectation value after sewing. We therefore test a narrower claim: whether independent subcircuit stopping preserves that reconstructed observable relative to the corresponding full-budget cutting workflow while reducing the number of executed shots. We do not test preservation of the complete reconstructed probability distribution.

4 Experimental Evaluation

We evaluate the proposed approach on a benchmark set of Clifford+ T circuits⁵. Clifford+ T is a standard gate set used to construct quantum circuits; here it provides controlled benchmark circuits with different sizes.

For each circuit size $q \in \{10, \dots, 16\}$, we generate 10 circuits, obtaining a total of 70 benchmark circuits. Each circuit is executed on four Qiskit fake backends: `fake_torino`, `fake_sherbrooke`, `fake_kawasaki`, and `fake_kyoto`. Fake backends are simulator models that reproduce backend characteristics and

⁵ Replication repository: https://github.com/alebocci/cut_and_stable_shots.

noise for controlled, reproducible experiments [3]. We consider three global shot-budget tiers: $B \in \{5\,000, 10\,000, 20\,000\}$.

The benchmark consists of seeded synthetic Clifford+T circuits with a bipartite structure and a mix of intra- and cross-partition CNOT gates. Each circuit is cut automatically using PennyLane’s KaHyPar-based pipeline, retaining instances that produce valid multi-fragment decompositions under the configured size and cutting constraints. For each circuit, we record structural information such as qubit count, gate counts, fragment sizes, number of cuts, and cutting variations. This construction provides a controlled and reproducible workload for evaluating circuit-cutting execution, while not claiming to represent the full range of realistic quantum applications.

All circuits are cut using the same automated cutting rule before execution. In our experiments, the cutting backend is configured to produce subcircuits that fit an artificially constrained target size, so that circuit cutting is exercised even for circuits with 10–16 qubits. This choice is deliberate: the goal is not to show that these benchmark circuits cannot run on the selected fake backends, but to evaluate adaptive shot control inside a controlled circuit-cutting workflow. These experiments therefore do not show that the selected circuits require cutting on those backends; cutting is imposed to create the multi-subcircuit execution setting in a controlled way. After cutting, each fragment may generate multiple execution variants, corresponding to the measurement and preparation configurations introduced by the cutting procedure. We treat each generated variant as an independent execution unit for shot allocation and adaptive stopping.

For every circuit–backend–budget configuration, we compare five execution modes:

- VAN: vanilla execution without circuit cutting.
- CC-SUB: circuit cutting with shots allocated proportionally to the number of generated subcircuits.
- CC-QUB: circuit cutting with shots allocated proportionally to the number of qubits of each subcircuit.
- SS-SUB: circuit cutting with STABLESHOTS using the CC-SUB allocation as the maximum budget for each subcircuit execution.
- SS-QUB: circuit cutting with STABLESHOTS using the CC-QUB allocation as the maximum budget for each subcircuit execution.

The goal of this comparison is not to determine whether CC-SUB or CC-QUB is the best static allocation policy. Instead, these policies provide two different fixed-budget baselines for assessing the effect of adaptive execution control. For each static policy, we compare the corresponding STABLESHOTS variant against its full-budget counterpart: SS-SUB against CC-SUB, and SS-QUB against CC-QUB. This pairwise comparison allows us to evaluate whether online stopping reduces shot consumption without substantially altering the observable estimate produced by the underlying fixed-budget cutting workflow.

For a global budget B , CC-SUB distributes shots uniformly across the generated execution units, while CC-QUB distributes shots proportionally to the

qubit count of the fragment associated with each execution unit. Here, an execution unit is one generated subcircuit variant submitted for measurement after cutting. In both cases, fractional allocations are rounded to integer shot counts while preserving the total allocated budget B . The two static cutting policies, CC-SUB and CC-QUB, are inspired by prior shot-allocation strategies [4] based on structural properties of the cut workload.

In all experiments, StableShots is used with batch size $b = 50$, lookback window $\ell = 3$, stabilization threshold $\epsilon = 0.05$, and stability parameter $k = 5$. These values were chosen in an initial calibration phase and then fixed for every circuit, backend, budget, and allocation policy. This keeps the evaluation focused on online stopping rather than per-instance tuning.

Lowering ϵ makes the distance threshold stricter, while increasing k requires stability to persist for more checks; either change tends to delay stopping. Increasing ℓ compares the current estimate with a longer history and can make the criterion more conservative. The batch size b sets the decision granularity: smaller batches allow earlier checks but require more execution rounds, whereas larger batches reduce the number of checks but can consume extra shots after an execution unit has already stabilized.

Overall, the evaluation considers 70 Clifford+ T circuits, 4 backend models, 3 budget tiers, and 5 execution modes, for a total of 4 200 circuit-backend-budget-mode configurations. We compare the methods in terms of accuracy, measuring the estimation error, and shot consumption, measuring the overall number of shots executed.

The stopping rule monitors the empirical distribution of each execution unit, but our evaluation is at the observable level. PennyLane’s `cut_circuit` transform returns the expectation value of the selected observable after sewing, so we can assess how online stopping affects that reconstructed value, not the complete reconstructed distribution. For each q -qubit circuit, we consider the observable $O = Z \otimes I^{\otimes(q-1)}$, corresponding to a Pauli- Z measurement on the first qubit and identity operations on the remaining qubits. We compute for each mode M the estimated expectation value $\widehat{\langle O \rangle}_M$ and report

$$e(M) = \left| \widehat{\langle O \rangle}_M - \langle O \rangle^* \right|,$$

where $\langle O \rangle^*$ is obtained from a noiseless simulator. We also compare each STABLESHOTS mode with its corresponding full-budget cutting baseline: SS-SUB against CC-SUB, and SS-QUB against CC-QUB. Similar errors in these pairwise comparisons indicate that the adaptive modes achieve error levels close to their matched fixed-budget baselines for the evaluated observable, while the shot-saving metric quantifies the reduction in shot consumption.

4.1 Results

Figure 3 reports the mean absolute error on the estimated expectation value of O and the percentage of shots saved across the three global budget tiers. Each

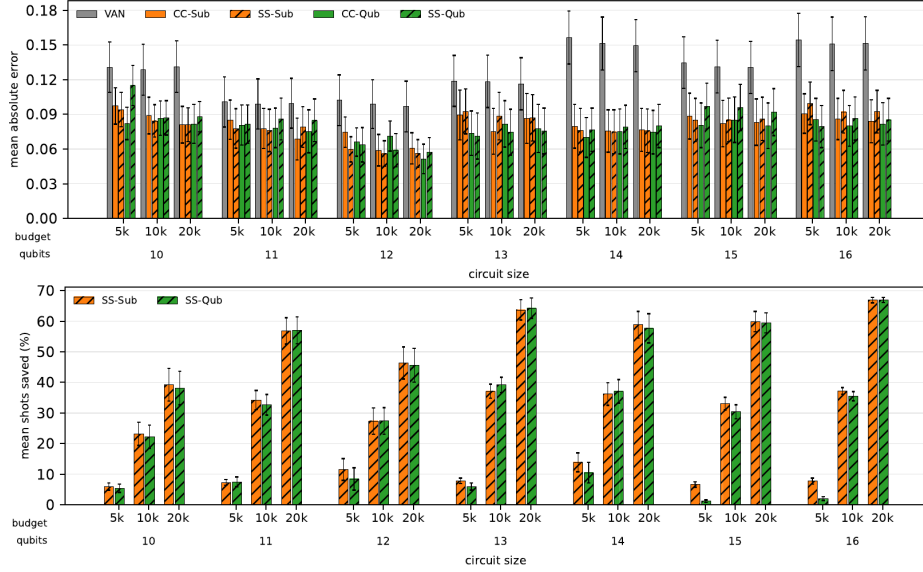


Fig. 3. Experimental results grouped by global shot budget. Bars report the mean over the 10 Clifford+ T circuits and the four fake backends; whiskers report the standard error of the mean (SEM). Hatched bars denote the StableShots configurations.

bar represents the mean over 10 circuits and 4 fake backends for a mode, while whiskers report the standard error of the mean (SEM).

The upper panel shows the error: how close each execution mode is to the noiseless reference expectation value. The STABLESHOTS variants remain close to their corresponding fixed-budget baselines. In particular, SS-SUB closely follows CC-SUB, with both average errors around 0.081. Similarly, SS-QUB remains close to CC-QUB, with average errors of about 0.082 and 0.077, respectively. These results indicate that STABLESHOTS introduces only limited accuracy loss with respect to full-budget circuit cutting. Overall, all circuit-cutting modes achieve lower error than the vanilla execution mode.

The lower panel reports the percentage of shots saved by STABLESHOTS. For the smallest budget, $B = 5k$, the saving is limited, about 7% on average, suggesting that most subcircuits require a large fraction of the available budget to stabilize. As the budget increases, redundant execution becomes more visible: the average saving rises to about 32% for $B = 10k$ and to about 56% for $B = 20k$. The largest savings are observed for the 20k budget, reaching approximately 64% for 13-qubit circuits and 67% for 16-qubit circuits.

Table 1 complements the descriptive results in Fig. 3 with the two matched comparisons used throughout the evaluation: SS-SUB against CC-SUB, and SS-QUB against CC-QUB. The CC and SS error columns report mean absolute error over the 21 circuit-size-budget groups, and the median error difference Δe

Table 1. Statistical comparison of each StableShots configuration with its matched full-budget baseline. Δe denotes the paired error difference, computed as SS minus CC; p is the p -value of the Wilcoxon signed-rank test.

Pair	CC error	SS error	Median Δe	p	Median savings	IQR
SS-SUB/CC-SUB	0.081	0.081	+0.0004	0.938	34.2%	11.5–46.3%
SS-QUB/CC-QUB	0.077	0.082	+0.0053	0.156	32.7%	8.5–45.6%

is computed as SS minus CC. Across the same groups, the median shot savings are 34.2% [11.5–46.3%] for SS-SUB and 32.7% [8.5–45.6%] for SS-QUB.

For the paired statistical comparison, we use a Wilcoxon signed-rank test after averaging the three budgets within each qubit size, leaving seven paired observations for each allocation policy. The median error difference is +0.0004 for SS-SUB versus CC-SUB ($p = 0.938$) and +0.0053 for SS-QUB versus CC-QUB ($p = 0.156$). Neither test rejects a zero median difference at $\alpha = 0.05$. With seven grouped pairs, the test checks for a systematic shift at the aggregate level; it does not establish equivalence or guarantee accuracy preservation for every individual circuit.

For the evaluated observable and configurations, the adaptive modes achieve error levels close to their corresponding full-budget baselines while using fewer shots.

5 Concluding Remarks

This paper addressed shot allocation in circuit cutting from a service-oriented execution-control perspective. Instead of treating the shot budget as a fixed amount that must always be fully consumed, the proposed workflow adds an adaptive execution-control layer between cutting and sewing. This layer monitors subcircuit execution online and stops subcircuits whose empirical output distributions have stabilized, while leaving the cut selection and reconstruction procedures unchanged.

On the evaluated Clifford+ T benchmarks, the adaptive modes produce error levels close to their matched full-budget baselines while reducing shot use by more than 25% on average, with savings reaching about 67%. The comparison in Table 1 is group-level and does not establish per-circuit equivalence. These results apply to the tested observable and configurations, not to the full reconstructed distribution.

The evaluation uses synthetic 10-16 qubit circuits, forced cutting, Qiskit fake backends, and one fixed STABLESHOTS parameter setting. Broader workloads and real quantum hardware may change both stabilization behavior and the relation between shot savings, latency, and cost. The reported savings should therefore be read as results for the evaluated configurations rather than as general guarantees.

Future work will test additional circuit families, observables, and real quantum hardware; study parameter sensitivity; and evaluate coordinated budget policies, including redistribution of unused shots among subcircuits. We also plan to examine more closely how subcircuit-level stabilization affects final reconstruction error.

Disclosure of Interests The authors have no competing interests to declare that are relevant to the content of this article.

Artifact Availability and Reproducibility Statement Replication materials are available at https://github.com/alebocci/cut_and_stable_shots. The repository contains the circuit-generation and benchmark-execution scripts, STABLESHOTS configuration files, generated circuits and summaries, experiment logs and results, and the analysis notebook used to aggregate accuracy and shot-consumption results.

References

1. Bisicchia, G., Clemente, G., García-Alonso, J., Murillo, J.M., D’Elia, M., Brogi, A.: Distributing quantum computations, shot-wise. *Future Internet* 17(11), 507 (2025). <https://doi.org/10.3390/fi17110507>
2. Bisicchia, G., Alvarado-Valiente, J., Romero-Álvarez, J., García-Alonso, J., Murillo, J.M., Brogi, A.: Maximizing quantum hardware utilization via multiprogramming circuits and shot-wise distribution. *Information and Software Technology* 191, 108005 (2026). <https://doi.org/10.1016/j.infsof.2025.108005>
3. Bisicchia, G., Bocci, A., García-Alonso, J., Murillo, J.M., Brogi, A.: QSimBench: An execution-level benchmark suite for quantum software engineering. In: 2025 IEEE International Conference on Quantum Computing and Engineering (QCE), pp. 175–180. IEEE (2025). <https://doi.org/10.1109/QCE65121.2025.10315>
4. Bisicchia, G., Bocci, A., García-Alonso, J., Murillo, J.M., Brogi, A.: Cut&Shoot: Distributed execution of quantum circuit fragments. *IEEE Transactions on Quantum Engineering* 7, 1–14 (2026). <https://doi.org/10.1109/TQE.2026.3675340>
5. Bisicchia, G., Bocci, A., Pimentel, E., Brogi, A.: How many shots are enough for a quantum circuit? *ACM Transactions on Quantum Computing* (2026). <https://doi.org/10.1145/3841468>
6. Bocci, A., Bisicchia, G., Pimentel, E., Brogi, A.: Formalising the incremental execution of static quantum circuits. *Journal of Logical and Algebraic Methods in Programming* 152, 101174 (2027). <https://doi.org/10.1016/j.jlamp.2026.101174>
7. Bisicchia, G., Bocci, A., Pimentel, E., Brogi, A.: StableShots: Adaptive shot control for quantum circuits. In: IEEE International Conference on Quantum Computing and Engineering (QCE 2026). IEEE (2026). In press
8. Brenner, L., Piveteau, C., Sutter, D.: Optimal wire cutting with classical communication. *IEEE Transactions on Information Theory* 71(10), 7742–7752 (2025). <https://doi.org/10.1109/TIT.2025.3601047>
9. Chen, P.H., Chiou, D.W., Chen, B.H., Jiang, J.H.R.: Enhanced quantum circuit cutting framework for sampling overhead reduction. *arXiv:2412.17704v3* (2024)

10. García-Alonso, J., Moguel, E., Alvarado-Valiente, J., Romero-Álvarez, J., Aparicio-Morales, Á.M., Murillo, J.M., Caverio, F.J., Romero-Flores, A., Marquez-Chamorro, A.E., Parejo, J.A., Ruiz-Cortés, A., Bisicchia, G., Bocci, A., Brogi, A.: Rethinking services in the quantum age: The SOQ paradigm. *ACM Transactions on Software Engineering and Methodology* 35(9), 1–46 (2026). <https://doi.org/10.1145/3805807>
11. Gu, A., Lowe, A., Dub, P.A., Coles, P.J., Arrasmith, A.: Adaptive shot allocation for fast convergence in variational quantum algorithms. *arXiv:2108.10434* (2021)
12. Kim, Y., Jang, E., Kim, H., Choi, S., Lee, C., Kim, D., Kyoung, W., Shin, K., Ro, W.W.: Distribution-adaptive dynamic shot optimization for variational quantum algorithms. *Physical Review Research* 7, 043253 (2025). <https://doi.org/10.1103/PhysRevResearch.7.043253>
13. Kübler, J.M., Arrasmith, A., Cincio, L., Coles, P.J.: An adaptive optimizer for measurement-frugal variational algorithms. *Quantum* 4, 263 (2020). <https://doi.org/10.22331/q-2020-05-11-263>
14. Liang, S., Zhu, L., Liu, X., Yang, C., Li, X.: Artificial-intelligence-driven shot reduction in quantum measurement. *Chemical Physics Reviews* 5, 041403 (2024). <https://doi.org/10.1063/5.0219663>
15. Lowe, A., Medvidović, M., Hayes, A., O’Riordan, L.J., Bromley, T.R., Arrazola, J.M., Killoran, N.: Fast quantum circuit cutting with randomized measurements. *Quantum* 7, 934 (2023). <https://doi.org/10.22331/q-2023-03-02-934>
16. Peng, T., Harrow, A.W., Ozols, M., Wu, X.: Simulating large quantum circuits on a small quantum computer. *Physical Review Letters* 125(15), 150504 (2020). <https://doi.org/10.1103/PhysRevLett.125.150504>
17. Perlin, M.A., Saleem, Z.H., Suchara, M., Osborn, J.C.: Quantum circuit cutting with maximum-likelihood tomography. *npj Quantum Information* 7, 64 (2021). <https://doi.org/10.1038/s41534-021-00390-6>
18. Piveteau, C., Sutter, D.: Circuit knitting with classical communication. *IEEE Transactions on Information Theory* 70(4), 2734–2745 (2024). <https://doi.org/10.1109/TIT.2023.3310797>
19. Preskill, J.: Quantum computing in the NISQ era and beyond. *Quantum* 2, 79 (2018). <https://doi.org/10.22331/q-2018-08-06-79>
20. Schmitt, L., Piveteau, C., Sutter, D.: Cutting circuits with multiple two-qubit unitaries. *Quantum* 9, 1634 (2025). <https://doi.org/10.22331/q-2025-02-18-1634>
21. Tang, W., Tomesh, T., Suchara, M., Larson, J., Martonosi, M.: CutQC: Using small quantum computers for large quantum circuit evaluations. In: *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pp. 473–486 (2021). <https://doi.org/10.1145/3445814.3446758>